CEFET/RJ

DATA VISUALIZATION

Eduardo Ogasawara
eogasawara@ieee.org
https://eic.cefet-rj.br/~eogasawara

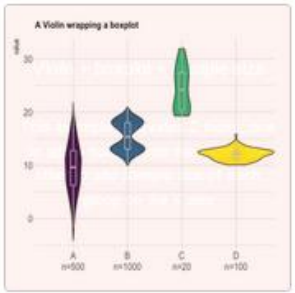
Plotting charts

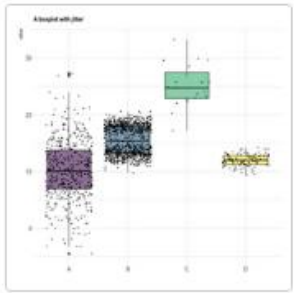
- R comes with basic plot functions
  - They are easy to operate for simple charts, but they are limited
- ggplot2 is a system for declaratively creating graphics, based on The Grammar of Graphics*.
- R for Data Science** is designed to give you a comprehensive introduction to ggplot2
  - The Data Visualization and Graphics for communication chapters

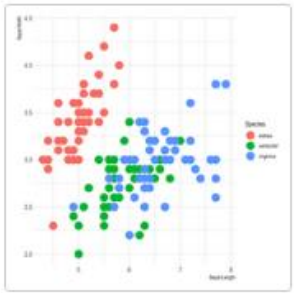
```
# The easiest way to get ggplot2 is to install the whole tidyverse:
# install.packages("tidyverse")
# Alternatively, install just ggplot2:
# install.packages("ggplot2")
```

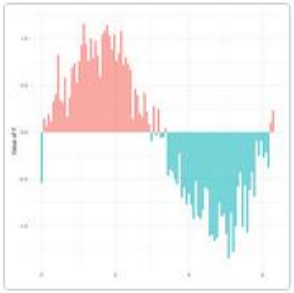
(*) https://www.amazon.com/Grammar-Graphics-Statistics-Computing/dp/0387245448
(**) https://r4ds.had.co.nz/

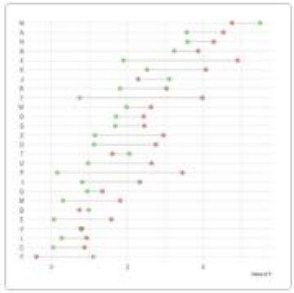
Examples of ggplot2 charts

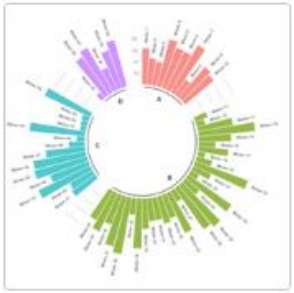


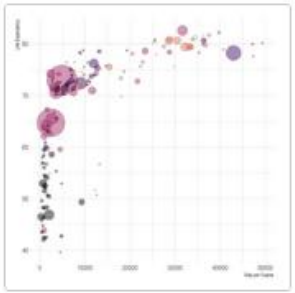


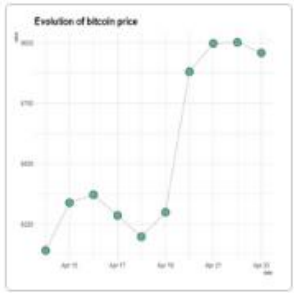


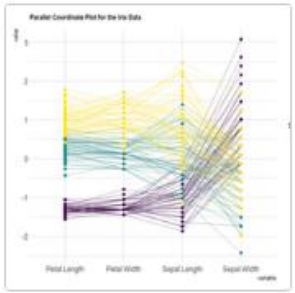


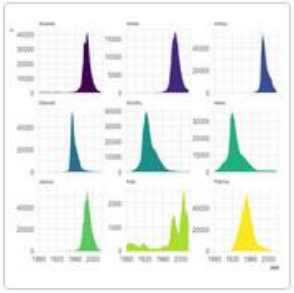


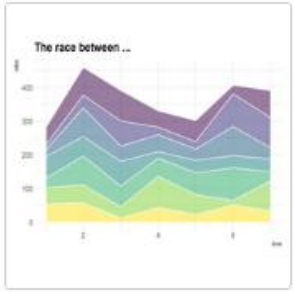












https://www.r-graph-gallery.com/ggplot2-package.html

ggplot2 cheat sheet

Data Visualization with ggplot2 : : CHEAT SHEET

Basics

ggplot2 is based on the **grammar of graphics**, the idea that you can build every graph from the same components: a **data** set, a **coordinate system**, and geoms—visual marks that represent data points.

To display values, map variables in the data to visual properties of the geom (**aesthetics**) like **size**, **color**, and **x** and **y** locations.

Complete the template below to build a graph.

```
ggplot (data = <DATA>) +                                  required
<GEOM_FUNCTION> (mapping = aes(<MAPPINGS>),
 stat = <STAT>, position = <POSITION> ) +                 Not required, sensible defaults supplied
<COORDINATE_FUNCTION> +
<FACET_FUNCTION> +
<SCALE_FUNCTION> +
<THEME_FUNCTION>
```

ggplot(data = mpg, **aes**(x = cty, y = hwy)) Begins a plot that you finish by adding layers to. Add one geom function per layer.

aesthetic mappings | data | geom

qplot(x = cty, y = hwy, data = mpg, geom = "point") Creates a complete plot with given data, geom, and mappings. Supplies many useful defaults.

last_plot() Returns the last plot

ggsave("plot.png", width = 5, height = 5) Saves last plot as 5' x 5' file named "plot.png" in working directory. Matches file type to file extension.

Geoms

Use a geom function to represent data points, use the geom's aesthetic properties to represent variables. Each function returns a layer.

GRAPHICAL PRIMITIVES

a <- ggplot(economics, aes(date, unemploy))
b <- ggplot(seals, aes(x = long, y = lat))

a + geom_blank()
(Useful for expanding limits)

b + geom_curve(aes(yend = lat + 1, xend=long+1),curvature=1) - x, xend, y, yend, alpha, angle, color, curvature, linetype, size

a + geom_path(lineend="butt", linejoin="round", linemitre=1)
x, y, alpha, color, group, linetype, size

a + geom_polygon(aes(group = group))
x, y, alpha, color, fill, group, linetype, size

b + geom_rect(aes(xmin = long, ymin=lat, xmax= long + 1, ymax = lat + 1)) - xmax, xmin, ymax, ymin, alpha, color, fill, linetype, size

a + geom_ribbon(aes(ymin=unemploy - 900, ymax=unemploy + 900)) - x, ymax, ymin, alpha, color, fill, group, linetype, size

LINE SEGMENTS

common aesthetics: x, y, alpha, color, linetype, size

b + geom_abline(aes(intercept=0, slope=1))
b + geom_hline(aes(yintercept = lat))
b + geom_vline(aes(xintercept = long))

b + geom_segment(aes(yend=lat+1, xend=long+1))
b + geom_spoke(aes(angle = 1:1155, radius = 1))

ONE VARIABLE continuous

c <- ggplot(mpg, aes(hwy)); c2 <- ggplot(mpg)

c + geom_area(stat = "bin")
x, y, alpha, color, fill, linetype, size

c + geom_density(kernel = "gaussian")
x, y, alpha, color, fill, group, linetype, size, weight

c + geom_dotplot()
x, y, alpha, color, fill

c + geom_freqpoly() x, y, alpha, color, group, linetype, size

c + geom_histogram(binwidth = 5) x, y, alpha, color, fill, linetype, size, weight

c2 + geom_qq(aes(sample = hwy)) x, y, alpha, color, fill, linetype, size, weight

discrete

d <- ggplot(mpg, aes(fl))

d + geom_bar()
x, alpha, color, fill, linetype, size, weight

TWO VARIABLES

continuous x , continuous y

e <- ggplot(mpg, aes(cty, hwy))

e + geom_label(aes(label = cty), nudge_x = 1, nudge_y = 1, check_overlap = TRUE) x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

e + geom_jitter(height = 2, width = 2)
x, y, alpha, color, fill, shape, size

e + geom_point(), x, y, alpha, color, fill, shape, size, stroke

e + geom_quantile(), x, y, alpha, color, group, linetype, size, weight

e + geom_rug(sides = "bl"), x, y, alpha, color, linetype, size

e + geom_smooth(method = lm), x, y, alpha, color, fill, group, linetype, size, weight

e + geom_text(aes(label = cty), nudge_x = 1, nudge_y = 1, check_overlap = TRUE), x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

discrete x , continuous y

f <- ggplot(mpg, aes(class, hwy))

f + geom_col(), x, y, alpha, color, fill, group, linetype, size

f + geom_boxplot(), x, y, lower, middle, upper, ymax, ymin, alpha, color, fill, group, linetype, shape, size, weight

f + geom_dotplot(binaxis = "y", stackdir = "center"), x, y, alpha, color, fill, group

f + geom_violin(scale = "area"), x, y, alpha, color, fill, group, linetype, size, weight

discrete x , discrete y

g <- ggplot(diamonds, aes(cut, color))

g + geom_count(), x, y, alpha, color, fill, shape, size, stroke

continuous bivariate distribution

h <- ggplot(diamonds, aes(carat, price))

h + geom_bin2d(binwidth = c(0.25, 500))
x, y, alpha, color, fill, linetype, size, weight

h + geom_density2d()
x, y, alpha, colour, group, linetype, size

h + geom_hex()
x, y, alpha, colour, fill, size

continuous function

i <- ggplot(economics, aes(date, unemploy))

i + geom_area()
x, y, alpha, color, fill, linetype, size

i + geom_line()
x, y, alpha, color, group, linetype, size

i + geom_step(direction = "hv")
x, y, alpha, color, group, linetype, size

visualizing error

df <- data.frame(grp = c("A", "B"), fit = 4:5, se = 1:2)
j <- ggplot(df, aes(grp, fit, ymin = fit-se, ymax = fit+se))

j + geom_crossbar(fatten = 2)
x, y, ymax, ymin, alpha, color, fill, group, linetype, size

j + geom_errorbar(), x, ymax, ymin, alpha, color, group, linetype, size, width (also **geom_errorbarh()**)

j + geom_linerange()
x, ymin, ymax, alpha, color, group, linetype, size

j + geom_pointrange()
x, y, ymin, ymax, alpha, color, fill, group, linetype, shape, size

maps

data <- data.frame(murder = USArrests$Murder, state = tolower(rownames(USArrests)))
map <- map_data("state")
k <- ggplot(data, aes(fill = murder))

k + geom_map(aes(map_id = state), map = map) **+ expand_limits**(x = map$long, y = map$lat), map_id, alpha, color, fill, linetype, size

THREE VARIABLES

seals$z <- with(seals, sqrt(delta_long^2 + delta_lat^2)); l <- ggplot(seals, aes(long, lat))

l + geom_contour(aes(z = z))
x, y, z, alpha, colour, group, linetype, size, weight

l + geom_raster(aes(fill = z), hjust=0.5, vjust=0.5, interpolate=FALSE)
x, y, alpha, fill

l + geom_tile(aes(fill = z)), x, y, alpha, color, fill, linetype, size, width

RStudio® is a trademark of RStudio, Inc. • CC BY SA RStudio • info@rstudio.com • 844-448-1212 • rstudio.com • Learn more at **http://ggplot2.tidyverse.org** • ggplot2 3.1.0 • Updated: 2018-12

https://github.com/rstudio/cheatsheets/blob/master/data-visualization-2.1.pdf

ggplot2 cheat sheet

Stats

An alternative way to build a layer

A stat builds new variables to plot (e.g., count, prop).

Visualize a stat by changing the default stat of a geom function, **geom_bar(stat="count")** or by using a stat function, **stat_count(geom="bar")**, which calls a default geom to make a layer (equivalent to a geom function). Use **..name..** syntax to map stat variables to aesthetics.

c + stat_bin(binwidth = 1, origin = 10**)**
x, y | ..count.., ..ncount.., ..density.., ..ndensity..

c + stat_count(width = 1**) x, y, |** ..count.., ..prop..

c + stat_density(adjust = 1, kernel = "gaussian"**)**
x, y, | ..count.., ..density.., ..scaled..

e + stat_bin_2d(bins = 30, drop = T**)**
x, y, fill | ..count.., ..density..

e + stat_bin_hex(bins=30**) x, y, fill |** ..count.., ..density..

e + stat_density_2d(contour = TRUE, n = 100**)**
x, y, color, size | ..level..

e + stat_ellipse(level = 0.95, segments = 51, type = "t"**)**

l + stat_contour(aes(z = z)**) x, y, z, order |** ..level..

l + stat_summary_hex(aes(z = z), bins = 30, fun = max**)**
x, y, z, fill | ..value..

l + stat_summary_2d(aes(z = z), bins = 30, fun = mean**)**
x, y, z, fill | ..value..

f + stat_boxplot(coef = 1.5**) x, y |** ..lower.., ..middle.., ..upper.., ..width.. , ..ymin.., ..ymax..

f + stat_ydensity(kernel = "gaussian", scale = "area"**) x, y |** ..density.., ..scaled.., ..count.., ..n.., ..violinwidth.., ..width..

e + stat_ecdf(n = 40**) x, y |** ..x.., ..y..

e + stat_quantile(quantiles = c(0.1, 0.9), formula = y ~ log(x), method = "rq"**) x, y |** ..quantile..

e + stat_smooth(method = "lm", formula = y ~ x, se=T, level=0.95**) x, y |** ..se.., ..x.., ..y.., ..ymin.., ..ymax..

ggplot() + stat_function(aes(x = -3:3), n = 99, fun = dnorm, args = list(sd=0.5)**) x |** ..x.., ..y..

e + stat_identity(na.rm = TRUE**)**

ggplot() + stat_qq(aes(sample=1:100), dist = qt, dparam=list(df=5)**) sample, x, y |** ..sample.., ..theoretical..

e + stat_sum() x, y, size | ..n.., ..prop..

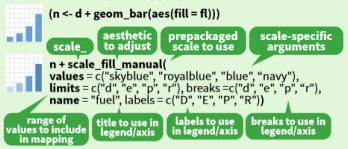
e + stat_summary(fun.data = "mean_cl_boot"**)**

h + stat_summary_bin(fun = "mean", geom = "bar"**)**

e + stat_unique()

Scales

Scales map data values to the visual values of an aesthetic. To change a mapping, add a new scale.



GENERAL PURPOSE SCALES

Use with most aesthetics

scale_*_continuous() - map cont' values to visual ones
scale_*_discrete() - map discrete values to visual ones
scale_*_identity() - use data values **as** visual ones
scale_*_manual(values = c()**)** - map discrete values to manually chosen visual ones
scale_*_date(date_labels = "%m/%d"), date_breaks = "2 weeks"**)** - treat data values as dates.
scale_*_datetime() - treat data x values as date times. Use same arguments as scale_x_date(). See ?strptime for label formats.

X & Y LOCATION SCALES

Use with x or y aesthetics (x shown here)

scale_x_log10() - Plot x on log10 scale
scale_x_reverse() - Reverse direction of x axis
scale_x_sqrt() - Plot x on square root scale

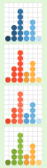
COLOR AND FILL SCALES (DISCRETE)

n <- d + geom_bar(aes(fill = fl)**)**

n + scale_fill_brewer(palette = "Blues"**)**
For palette choices:
RColorBrewer::display.brewer.all()

n + scale_fill_grey(start = 0.2, end = 0.8, na.value = "red"**)**

COLOR AND FILL SCALES (CONTINUOUS)



o <- c + geom_dotplot(aes(fill = ..x..))

o + scale_fill_distiller(palette = "Blues"**)**

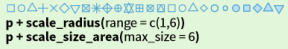
o + scale_fill_gradient(low="red", high="yellow"**)**

o + scale_fill_gradient2(low="red", high="blue", mid = "white", midpoint = 25**)**

o + scale_fill_gradientn(colours=topo.colors(6)**)**
Also: rainbow(), heat.colors(), terrain.colors(), cm.colors(), RColorBrewer::brewer.pal()

SHAPE AND SIZE SCALES

p <- e + geom_point(aes(shape = fl, size = cyl))
p + scale_shape() + scale_size()
p + scale_shape_manual(values = c(3:7)**)**



p + scale_radius(range = c(1,6)**)**
p + scale_size_area(max_size = 6**)**

Coordinate Systems

r <- d + geom_bar()

r + coord_cartesian(xlim = c(0, 5)**)**
xlim, ylim
The default cartesian coordinate system

r + coord_fixed(ratio = 1/2**)**
ratio, xlim, ylim
Cartesian coordinates with fixed aspect ratio between x and y units

r + coord_flip()
xlim, ylim
Flipped Cartesian coordinates

r + coord_polar(theta = "x", direction=1 **)**
theta, start, direction
Polar coordinates

r + coord_trans(ytrans = "sqrt"**)**
xtrans, ytrans, limx, limy
Transformed cartesian coordinates. Set xtrans and ytrans to the name of a window function.

π + coord_quickmap()
π + coord_map(projection = "ortho", orientation=c(41, -74, 0)**)**projection, xlim, ylim
Map projections from the mapproj package (mercator (default), azequalarea, lagrange, etc.)

Position Adjustments

Position adjustments determine how to arrange geoms that would otherwise occupy the same space.

s <- ggplot(mpg, aes(fl, fill = drv))

s + geom_bar(position = "dodge")
Arrange elements side by side

s + geom_bar(position = "fill")
Stack elements on top of one another, normalize height

e + geom_point(position = "jitter")
Add random noise to X and Y position of each element to avoid overplotting

e + geom_label(position = "nudge")
Nudge labels away from points

s + geom_bar(position = "stack")
Stack elements on top of one another

Each position adjustment can be recast as a function with manual **width** and **height** arguments
s + geom_bar(position = position_dodge(width = 1))

Themes

r + theme_bw()
White background with grid lines

r + theme_gray()
Grey background (default theme)

r + theme_dark()
dark for contrast

r + theme_classic()

r + theme_light()

r + theme_linedraw()

r + theme_minimal()
Minimal themes

r + theme_void()
Empty theme

Faceting

Facets divide a plot into subplots based on the values of one or more discrete variables.

t <- ggplot(mpg, aes(cty, hwy)) + geom_point()

t + facet_grid(cols = vars(fl))
facet into columns based on fl

t + facet_grid(rows = vars(year))
facet into rows based on year

t + facet_grid(rows = vars(year), cols = vars(fl))
facet into both rows and columns

t + facet_wrap(vars(fl))
wrap facets into a rectangular layout

Set **scales** to let axis limits vary across facets

t + facet_grid(rows = vars(drv), cols = vars(fl), scales = "free")
x and y axis limits adjust to individual facets
"free_x" - x axis limits adjust
"free_y" - y axis limits adjust

Set **labeller** to adjust facet labels

t + facet_grid(cols = vars(fl), labeller = label_both)

fl: c	fl: d	fl: e	fl: p	fl: r

t + facet_grid(rows = vars(fl), labeller = label_bquote(alpha ^ .(fl)**))**

α^c	α^d	α^e	α^p	α^r

Labels

t + labs(**x** = "New x axis label", **y** = "New y axis label",
title ="Add a title above the plot",
subtitle = "Add a subtitle below title",
caption = "Add a caption below plot",
<AES> = "New **<AES>** legend title"**)**

Use scale functions to update legend labels

t + annotate(geom = "text", x = 8, y = 9, label = "A"**)**

geom to place | manual values for geom's aesthetics

Legends

n + theme(legend.position = "bottom"**)**
Place legend at "bottom", "top", "left", or "right"

n + guides(fill = "none"**)**
Set legend type for each aesthetic: colorbar, legend, or none (no legend)

n + scale_fill_discrete(name = "Title", labels = c("A", "B", "C", "D", "E"))
Set legend title and labels with a scale function.

Zooming

Without clipping (preferred)

t + coord_cartesian(
xlim = c(0, 100), ylim = c(10, 20))

With clipping (removes unseen data points)

t + xlim(0, 100**) + ylim(**10, 20**)**

t + scale_x_continuous(limits = c(0, 100)**) + scale_y_continuous(**limits = c(0, 100)**)**

RStudio

RStudio® is a trademark of RStudio, Inc. • CC BY SA RStudio • info@rstudio.com • 844-448-1212 • rstudio.com • Learn more at **http://ggplot2.tidyverse.org** • ggplot2 3.1.0 • Updated: 2018-12

https://github.com/rstudio/cheatsheets/blob/master/data-visualization-2.1.pdf

Graphics

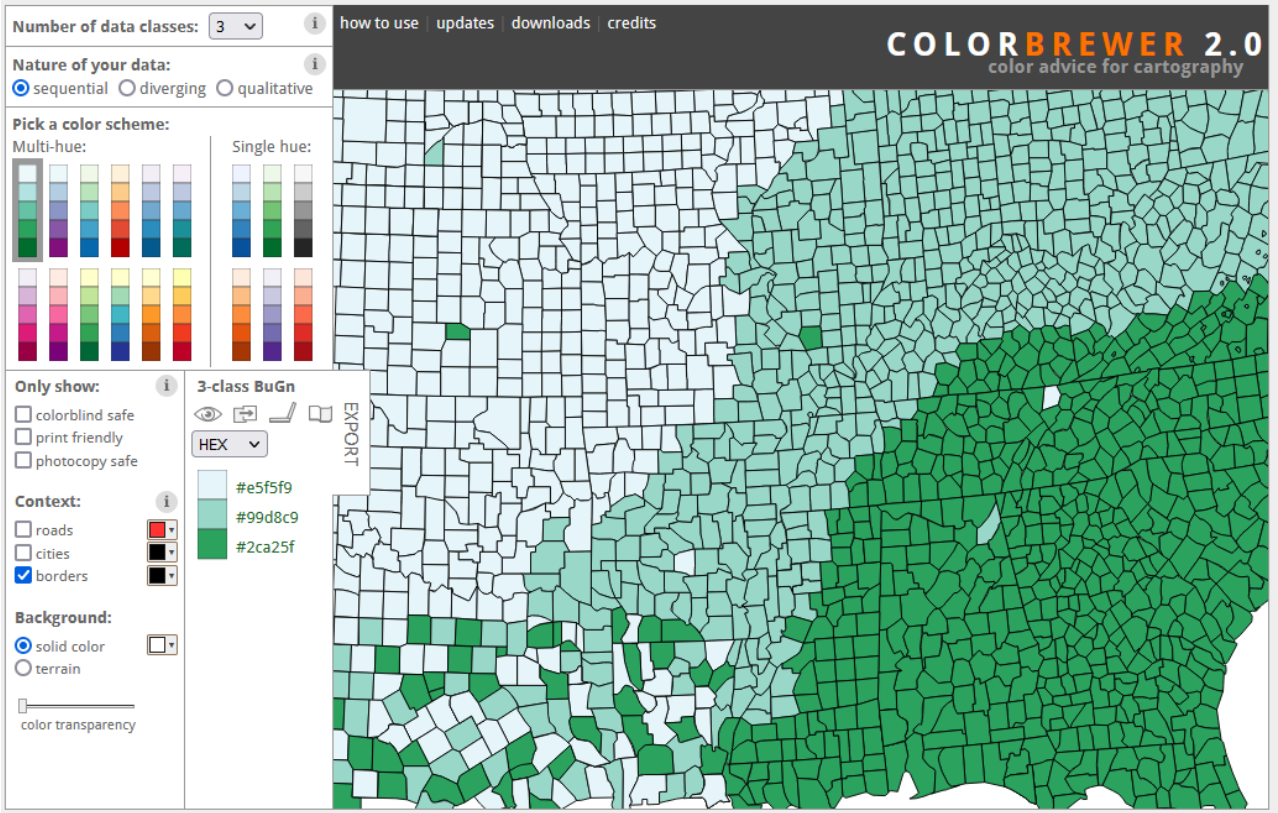
- The DAL Toolbox enables plotting charts encapsulating ggplot2
- It enables an easy startup while learning how to use ggplot2
- Most functions require a data.frame with two or more attributes.
  - In most cases, the first attribute is associated with the x-axis: x
  - The second is related to the y-axis: value
  - The following attributes might be continuation of values
    - Sometimes, the third might be a variable definition
  - The library DAL Toolbox is loaded using the source function

```
library(daltoolbox)
```

https://github.com/cefet-rj-dal/daltoolbox/tree/main/graphics

But before continuing, lets talk about colors…

- One thing very relevant while plotting charts is to preserve visual identity
- For that, the color brewer is an excellent tool to set up colors for your graphics using appropriate colors



https://colorbrewer2.org

Color Brewer R Package

- Take some time to look at how to use it in R

ColorBrewer: ColorBrewer palettes

In RColorBrewer: ColorBrewer Palettes

Description | Usage | Arguments | Details | Value | Note | Author(s) | Examples

Description

Creates nice looking color palettes especially for thematic maps

Usage

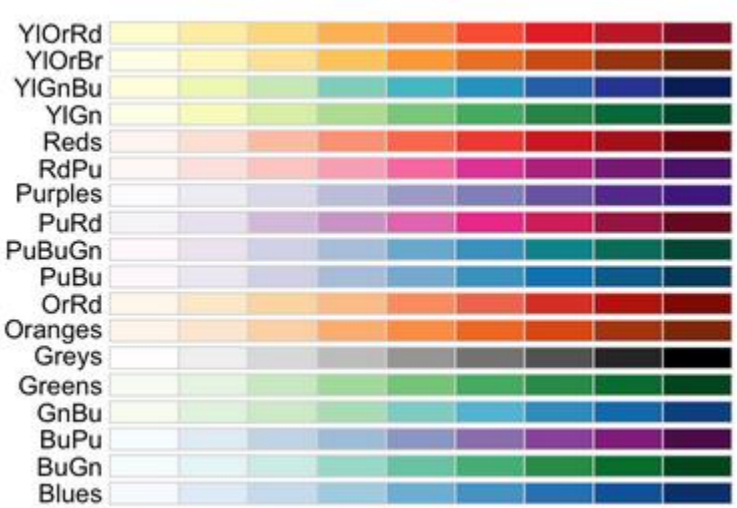
```
brewer.pal(n, name)
display.brewer.pal(n, name)
display.brewer.all(n=NULL, type="all", select=NULL, exact.n=TRUE,
colorblindFriendly=FALSE)
brewer.pal.info
```

https://rdrr.io/cran/RColorBrewer/man/ColorBrewer.html

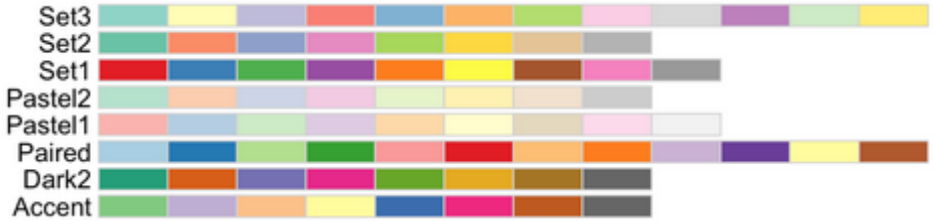
Basic code to use color brewer

```
#color palette
colors <- brewer.pal(4, 'Set1')

# This function is used only to set graphics size in this notebook.
plot.size(10, 5)

# setting the font size for all charts
font <- theme(text = element_text(size=16))
```

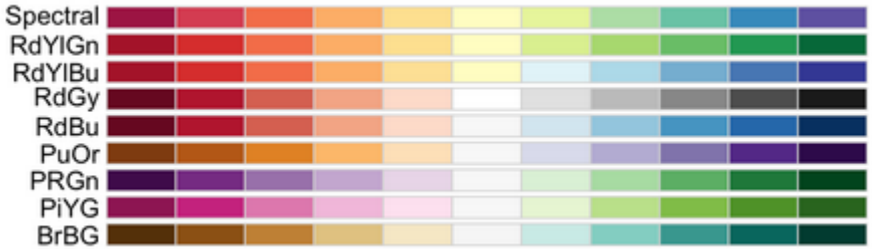
Sequential



Qualitative



Diverging



https://rdrr.io/cran/RColorBrewer/man/ColorBrewer.html

Iris dataset

```r
#iris dataset for the example
head(iris)
```

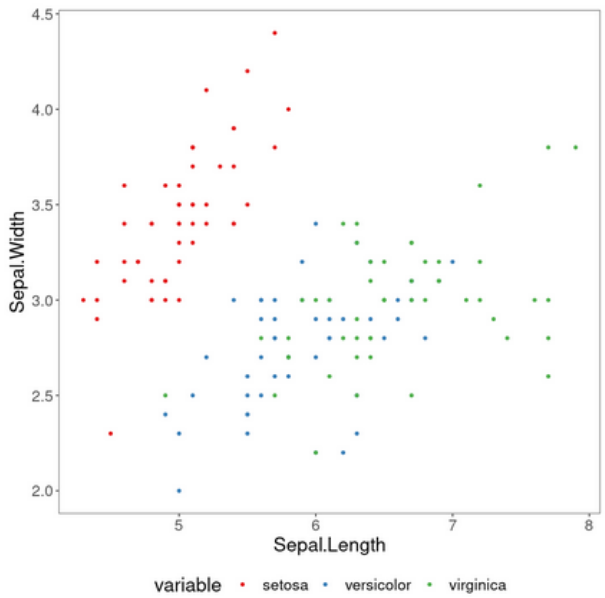
A data.frame: 6 × 5

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
	<dbl>	<dbl>	<dbl>	<dbl>	<fct>
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

Scatter plot

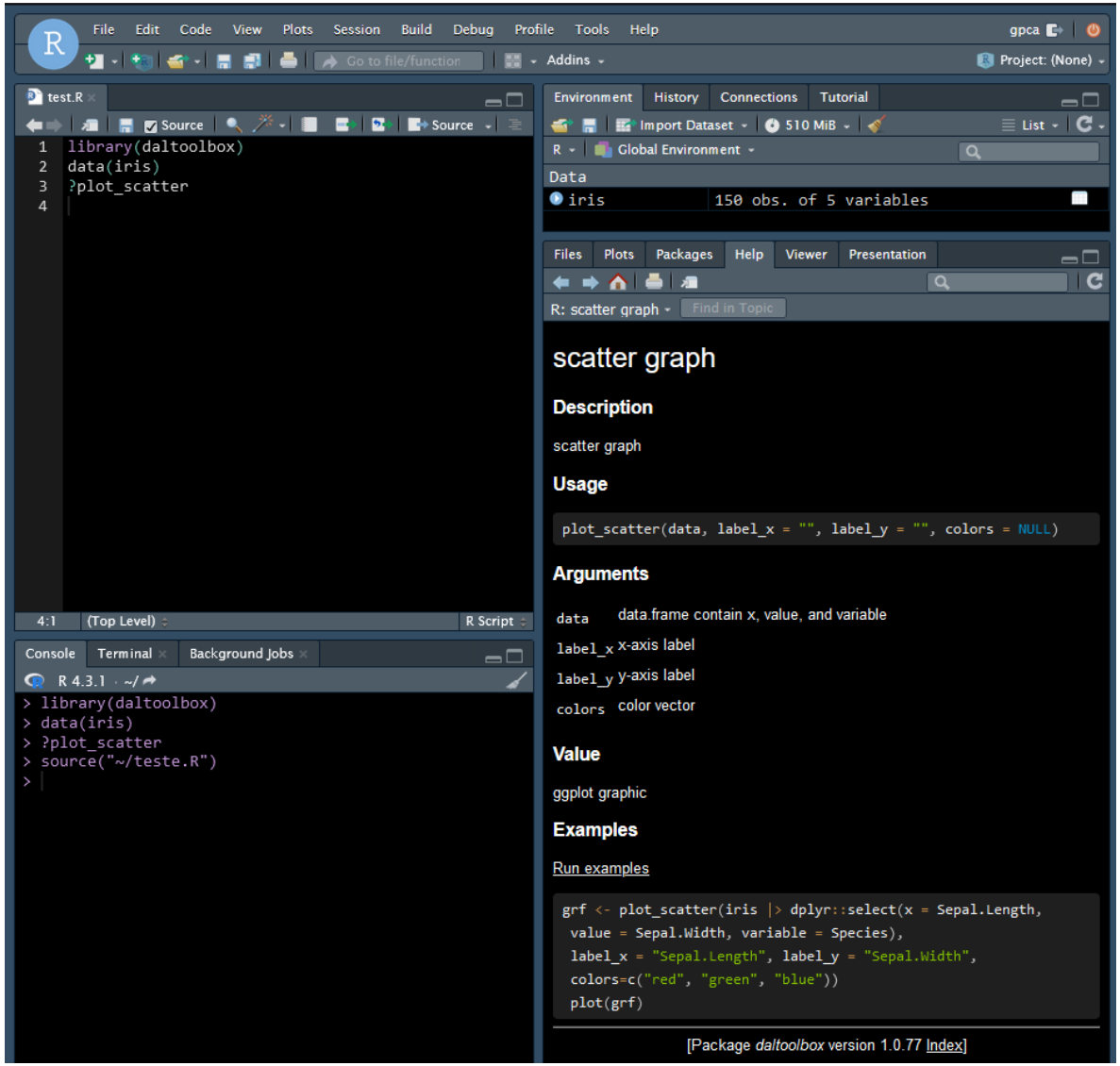
- A Scatter plot is used to display values for the typical relationship between independent and dependent variables
  - More information: https://en.wikipedia.org/wiki/Scatter_plot

```
load_library("dplyr")

grf <- plot_scatter(iris |> select(x = Sepal.Length, value = Sepal.Width,
                                   variable = Species),
                    label_x = "Sepal.Length",
                    label_y = "Sepal.Width",
                    colors=colors[1:3]) + font
plot(grf)
```



The first column of the dataset is the independent variable. The following variables are the dependent ones
The colors vectors should have the same size as the number of dependent column

Help for all functions



Summarizing Sepal.Length according to species

```
data <- iris |> group_by(Species) |> summarize(Sepal.Length=mean(Sepal.Length))
head(data)
```

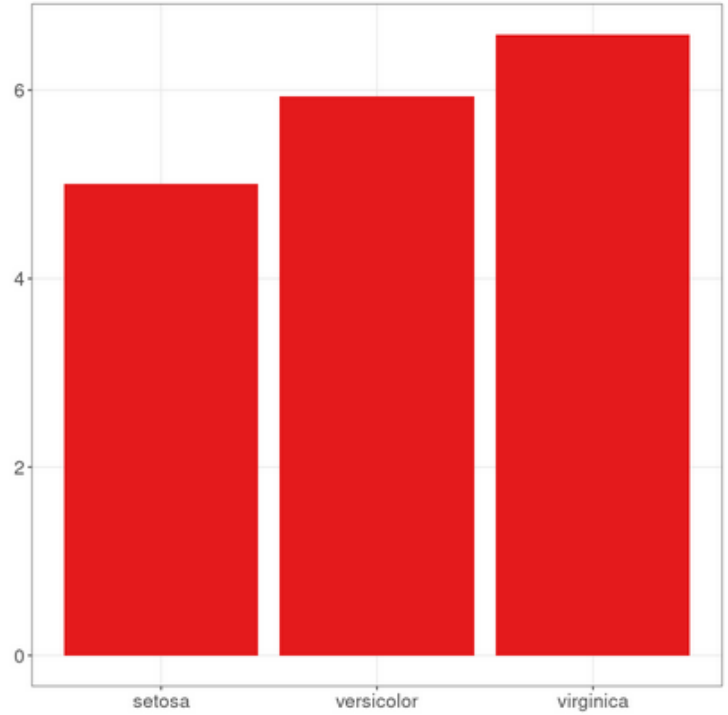
A tibble: 3 × 2

Species	Sepal.Length
<fct>	<dbl>
setosa	5.006
versicolor	5.936
virginica	6.588

Bar graph

- A bar graph is used to presents categorical data with rectangular bars with heights or lengths proportional to the values that they represent.
- More information: https://en.wikipedia.org/wiki/Bar_chart

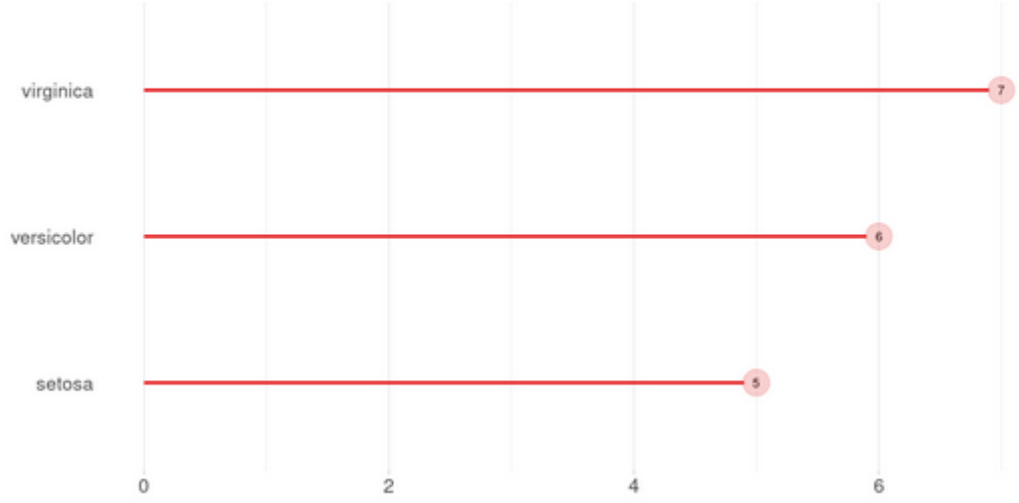
```
grf <- plot_bar(data, colors=colors[1]) + font
plot(grf)
```



Lollipop plot

- The lollipop graph has the same goal as a vertical bar graph

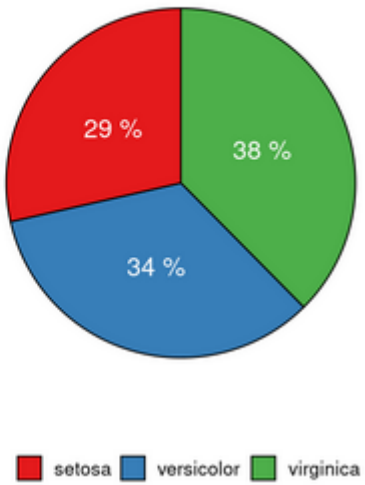
```
grf <- plot.lollipop(data, colors=colors[1], max_value_gap=0.1) + font
plot(grf)
```



Pie chart

- A pie chart is a circular statistical graphic, which is divided into slices to illustrate numerical proportion

```
grf <- plot.pieplot(data, colors=colors[1:3]) + font
plot(grf)
```



More information: https://en.wikipedia.org/wiki/Pie_chart

Summarizing attribute means

```r
load_library("dplyr")

data <- iris |> dplyr::select(-Species)
data <- sapply(data, mean)
data <- data.frame(name = names(data), value = data) |> dplyr::arrange(name)

head(data)
```

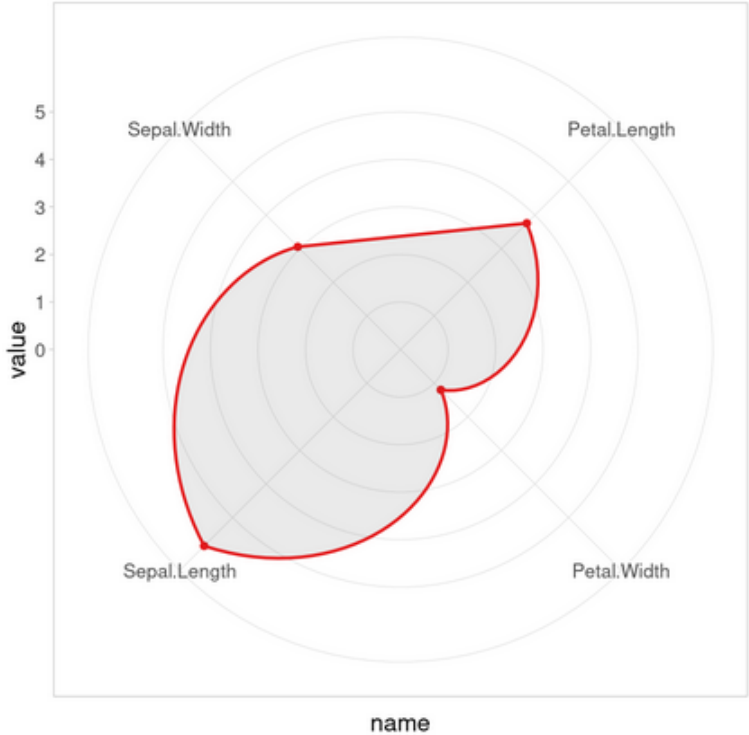
A data.frame: 4 × 2

	name	value
	<chr>	<dbl>
Petal.Length	Petal.Length	3.758000
Petal.Width	Petal.Width	1.199333
Sepal.Length	Sepal.Length	5.843333
Sepal.Width	Sepal.Width	3.057333

Radar chart

- A radar chart is a graphical method of displaying multivariate data containing three or more quantitative variables represented on axes starting from the same point

```
grf <- plot_radar(data, colors=colors[1]) + font
grf <- grf + ylim(0, NA)
plot(grf)
```



More information: https://en.wikipedia.org/wiki/Radar_chart

Summarizing Sepal.Length (mean and sd) according to species

```
data <- iris |> group_by(Species) |> summarize(mean=mean(Sepal.Length), sd=sd(Sepal.Length))
head(data)
```

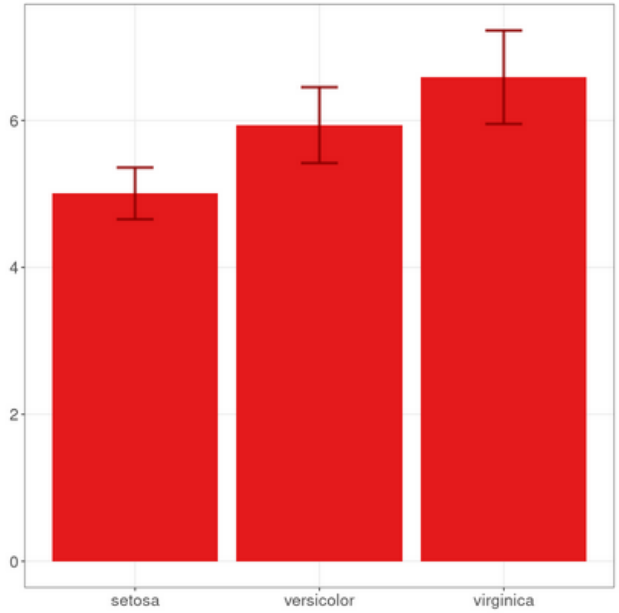
A tibble: 3 × 3

Species	mean	sd
<fct>	<dbl>	<dbl>
setosa	5.006	0.3524897
versicolor	5.936	0.5161711
virginica	6.588	0.6358796

Bar graph with error bars

- When bar graphs present an average behavior, it is possible to plot a dispersion around it using an error bar
- The error bar is added using geom_errorbar() function to a previously defined bar graph

```
grf <- plot_bar(data, colors=colors[1], alpha=1) + font
grf <- grf + geom_errorbar(aes(x=Species, ymin=mean-sd, ymax=mean+sd),
                           width=0.2, colour="darkred", alpha=0.8, size=1.1)
plot(grf)
```



Summarizing Sepal.Length and Sepal.Width according to species

```
data <- iris |> group_by(Species) |> summarize(Sepal.Length=mean(Sepal.Length), Sepal.Width=mean(Sepal.Width))
head(data)
```

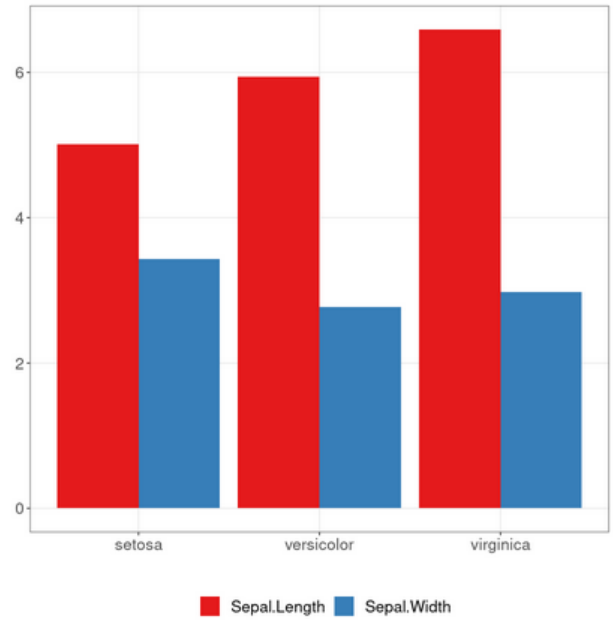
A tibble: 3 × 3

Species	Sepal.Length	Sepal.Width
<fct>	<dbl>	<dbl>
setosa	5.006	3.428
versicolor	5.936	2.770
virginica	6.588	2.974

Grouped bar

- Grouped bar charts and stacked bar charts are used to organize data into groups for each category
- In a grouped bar chart, for each categorical group, there are two or more bars. These bars are color-coded to represent a particular grouping

```
grf <- plot_groupedbar(data, colors=colors[1:2]) + font
plot(grf)
```

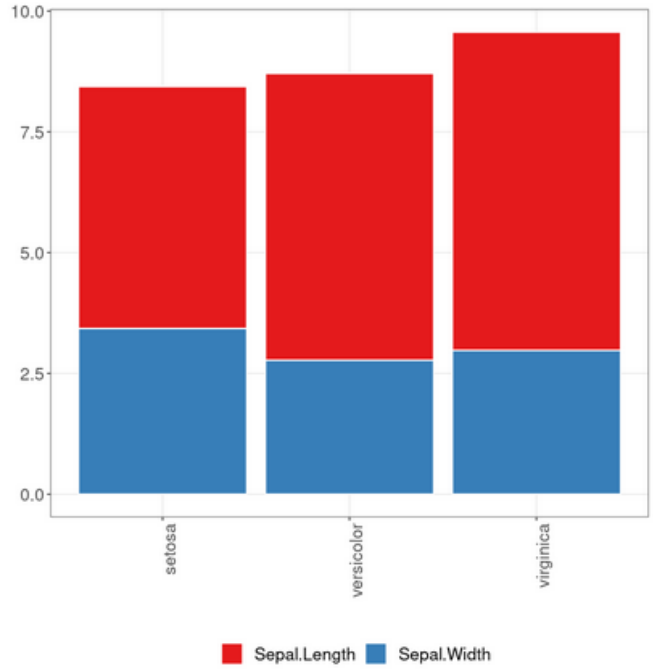


More information: https://en.wikipedia.org/wiki/Bar_chart#Grouped_or_stacked

Stacked-bars

- Alternatively, a stacked bar chart stacks bars that represent different groups on top of each other
- The height of the resulting bar shows the combined result of the groups

```
grf <- plot_stackedbar(data, colors=colors[1:2]) + font
grf <- grf + theme(axis.text.x = element_text(angle=90, hjust=1))
plot(grf)
```



More information: https://en.wikipedia.org/wiki/Bar_chart#Grouped_or_stacked

Time series

```r
x <- seq(0, 10, 0.25)
serie <- data.frame(x, sin=sin(x), cosine=cos(x)+5)
head(serie)
```

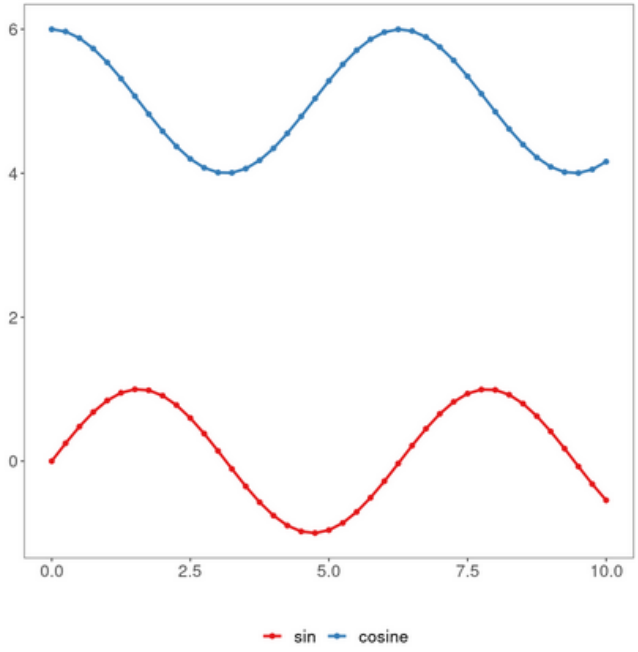
A data.frame: 6 × 3

	x	sin	cosine
	<dbl>	<dbl>	<dbl>
1	0.00	0.0000000	6.000000
2	0.25	0.2474040	5.968912
3	0.50	0.4794255	5.877583
4	0.75	0.6816388	5.731689
5	1.00	0.8414710	5.540302
6	1.25	0.9489846	5.315322

Line plot

- A line plot is a type of chart that displays information as a series of data points connected by straight line segments
- It is like a scatter plot except that their x-axis value orders the measurement points

```
grf <- plot_series(serie, colors=colors[1:2]) + font
plot(grf)
```



More information: https://en.wikipedia.org/wiki/Line_chart

Data Distribution Analysis

Examples using data distribution

- The following examples use random variables so that different data distribution can be better viewed

```r
# example: dataset to be plotted
example <- data.frame(exponential = rexp(100000, rate = 1),
                      uniform = runif(100000, min = 2.5, max = 3.5),
                      normal = rnorm(100000, mean=5))
head(example)
```

A data.frame: 6 × 3

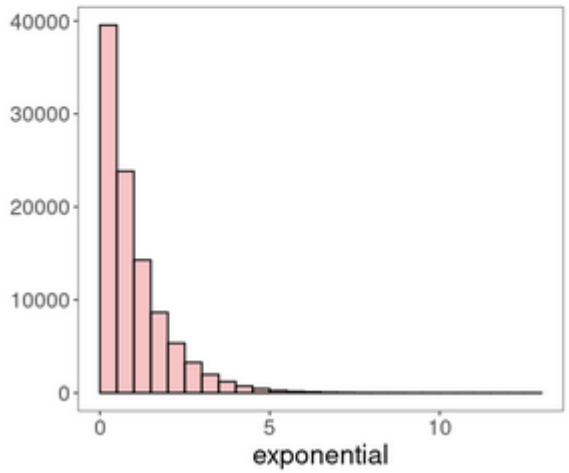
	exponential	uniform	normal
	<dbl>	<dbl>	<dbl>
1	0.94362792	3.443361	4.349481
2	0.58245523	3.206339	5.282097
3	1.63216251	2.565387	5.596367
4	0.03444139	2.885662	3.881086
5	0.13084487	3.273800	4.735605
6	0.16558101	2.759262	5.201384

Histogram

- Distribution of a single continuous variable by dividing the x axis into bins and counting the number of observations in each bin
  - Histograms (geom_histogram()) display the counts with bars

```
load_library("dplyr")

grf <- plot_hist(example |> select(exponential),
                 label_x = "exponential", color=colors[1]) + font
options(repr.plot.width=5, repr.plot.height=4)
plot(grf)
```

```
Using  as id variables
```



More information: ?geom_histogram (R documentation)

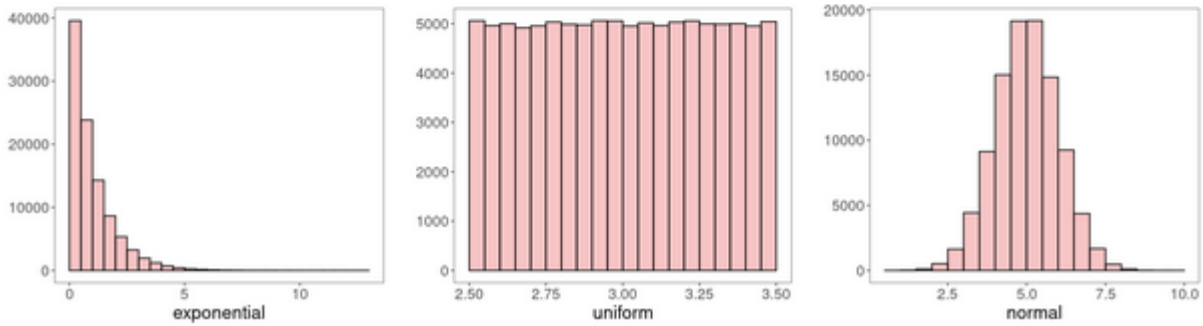
Arrangement of graphics

- Function grid.arrange is used to position previously computed charts

```
grfe <- plot_hist(example |> select(exponential),
                  label_x = "exponential", color=colors[1]) + font
grfu <- plot_hist(example |> select(uniform),
                  label_x = "uniform", color=colors[1]) + font
grfn <- plot_hist(example |> select(normal),
                  label_x = "normal", color=colors[1]) + font
```

```
Using  as id variables

Using  as id variables

Using  as id variables
```

```
load_library("gridExtra")

options(repr.plot.width=15, repr.plot.height=4)
grid.arrange(grfe, grfu, grfn, ncol=3)
```



More information: ?grid.arrange (R documentation)

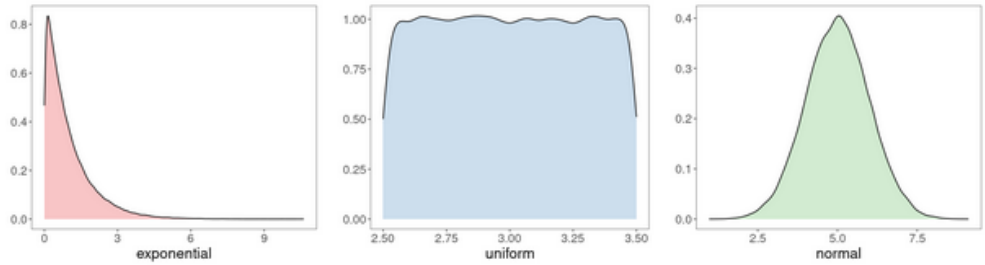
Density plot

- Computes and draws kernel density estimate, which is a smoothed version of the histogram
  - This is a useful alternative to the histogram for continuous data that comes from an underlying smooth distribution

```
load_library("dplyr")
grfe <- plot_density(example |> select(exponential),
                     label_x = "exponential", color=colors[1]) + font
grfu <- plot_density(example |> select(uniform),
                     label_x = "uniform", color=colors[2]) + font
grfn <- plot_density(example |> select(normal),
                     label_x = "normal", color=colors[3]) + font
```

```
Using  as id variables

Using  as id variables

Using  as id variables
```

```
load_library("gridExtra")
options(repr.plot.width=15, repr.plot.height=4)
grid.arrange(grfe, grfu, grfn, ncol=3)
```



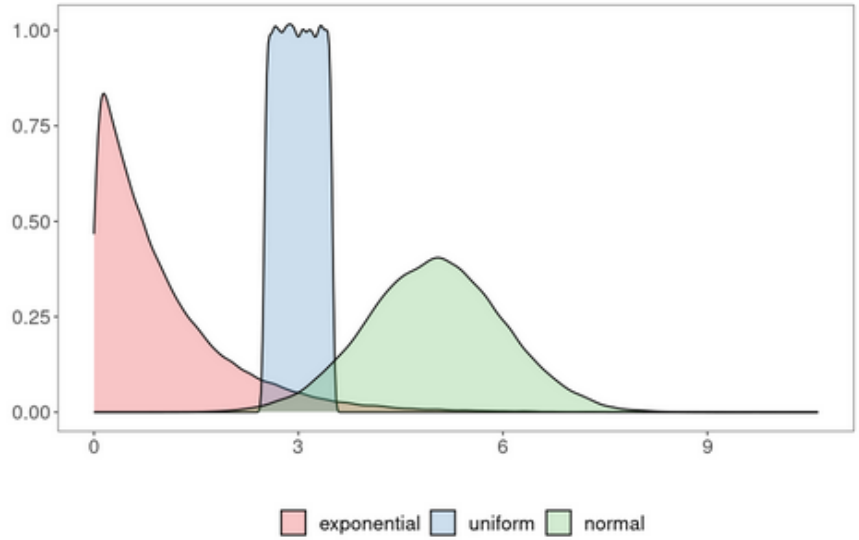
More information: ?geom_density (R documentation)

Density plot (combining in a single chart)

- Enables the comparison of different distributions

```
options(repr.plot.width=8, repr.plot.height=5)
grf <- plot_density(example, colors=colors[1:3]) + font
plot(grf)
```

```
Using  as id variables
```



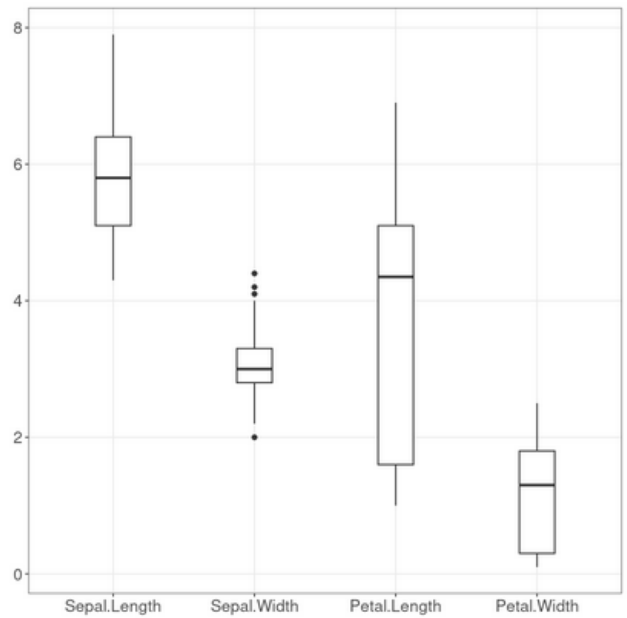
More information: ?geom_density (R documentation)

Box-plot

- In descriptive statistics, a box plot is a method for graphically depicting groups of numerical data through their quartiles.
  - Box plots may also have lines extending from the boxes (whiskers), indicating variability outside the upper and lower quartiles (outliers)

```
grf <- plot_boxplot(iris, colors="white") + font
plot(grf)
```

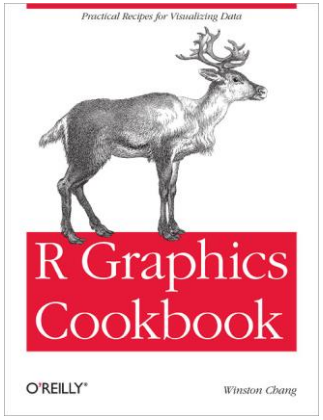
```
Using Species as id variables
```



More information: https://en.wikipedia.org/wiki/Box_plot

Main References

Código: https://github.com/eogasawara/datamining/blob/main/examples/3-DataVisualization.md



Further reading: R Graphics Cookbook, Chapters 1-6, 12

Slides e vídeos em: https://eic.cefet-rj.br/~eogasawara/data-mining/

- https://rstudio-education.github.io/hopr/index.html
- https://r-graphics.org